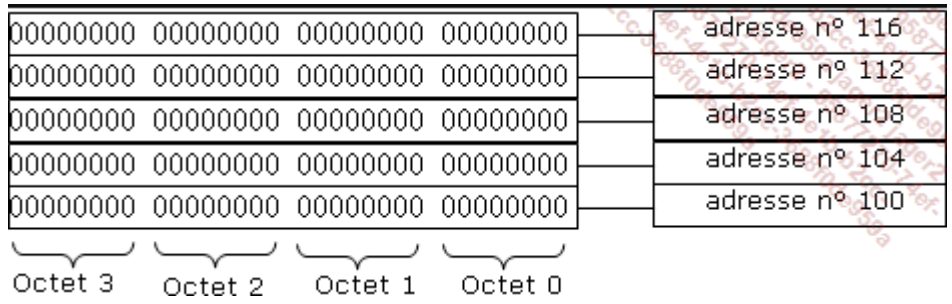
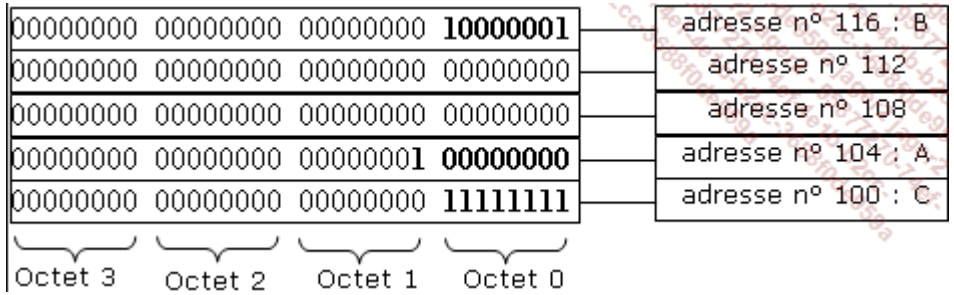
Type de donnée	Signification	Taille (en octets)	Plage de valeurs acceptée
char	Caractère (ASCII)	1	-128 à 127
unsigned char	Caractère non signé	1	0 à 255
short int	Entier court	2	-32 768 à 32 767
unsigned short int	Entier court non signé	2	0 à 65 535
int	Entier	2 (sur processeur 16 bits) 4 (sur processeur 32 bits)	-32 768 à 32 767 -2 147 483 648 à 2 147 483 647
unsigned int	Entier non signé	2 (sur processeur 16 bits) 4 (sur processeur 32 bits)	0 à 65 535 0 à 4 294 967 295
long int	Entier long	4	-2 147 483 648 à 2 147 483 647
unsigned long int	Entier long non signé	4	0 à 4 294 967 295
float	Flottant (réel)	4	$3.4*10-^{38}$ à $3.4*10^{38}$
double	Flottant double	8	$1.7*10-^{308}$ à $1.7*10^{308}$
long double	Flottant double long	10	$3.4*10-^{4932}$ à $3.4*10^{4932}$

Organisation de la mémoire RAM




```
char C=255;
int A=256, B=129;
```



Déclarer un pointeur dans un programme

Pour déclarer un pointeur, c'est-à-dire pour avoir un pointeur dans un programme, il faut :

- le type de l'objet sur lequel pointer,
- l'opérateur * (à gauche du nom du pointeur),
- un nom (identificateur) pour le pointeur.

```
char *c;            // déclare un pointeur sur char
int *i, *j;         // déclare deux pointeurs sur int
float *f1,*f2;      // déclare deux pointeurs sur float
```

c est un pointeur de type char* : il peut contenir des adresses de char.

i et j sont deux pointeurs de type int*, ils peuvent contenir des adresses de int.

f1 et f2 sont deux pointeurs de type float*, ils peuvent contenir des adresses de float.

Déclarer un pointeur dans un programme

Avec des structures

```
typedef struct player{    // définition du type player
    int x,y;
    int dx,dy;
}player;

player *p;                // déclare un pointeur player
```

p est un pointeur de type player* et p peut contenir des adresses de structures player.

Opérateur adresse : &

Il retourne l’adresse d’un objet

```
int k;
   &k          // cette expression vaut l'adresse mémoire de
               // la variable de k
```

soit k un int, &k est l’adresse qui pointe sur k.

Opérateur adresse : &

Le programme suivant permet d'afficher les adresses mémoire de plusieurs variables en utilisant l'opérateur & (et le format %p de printf() réservé pour afficher des valeurs de pointeur, c'est-à-dire des adresses mémoire) :

```c
#include <stdio.h>
#include <stdlib.h>

int main()
{
struct {
     int x,y;
     float dx,dy;
}s1;

int i,j;
char c;
    printf("l'adresse de c est : %p\n", &c);
    printf("l'adresse de i est : %p\n", &i);
    printf("l'adresse de j est : %p\n", &j);
    printf("l'adresse de s1 est : %p\n", &s1);
    return 0;
}
```

Pourquoi ces adresses ?

```
l'adresse de c est : 0x7fffc0d350a7
l'adresse de i est : 0x7fffc0d350ac
l'adresse de j est : 0x7fffc0d350a8
l'adresse de s1 est : 0x7fffc0d350b0
```

Faire de même sur le programme exo2 du TD1. Dessiner le contenu de la mémoire.

Opérateur étoile : *

Il permet d'accéder à l'objet via son adresse et de le modifier. Equivalent à « contenu de »

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
int i = 23;                // 1
int *ptr;

    ptr = &i;              // 2
    printf("%d, ",*ptr);

    *ptr = 55;             // 3
    printf("%d, ",i);

    i = 777;               // 4
    printf("%d.",*ptr);
    return 0;
}
```

(1) Déclarations de l'entier i qui vaut 23 et du pointeur d'entier ptr.

(2) L'adresse de i, c'est-à-dire l'expression &i, est affectée au pointeur ptr.
À partir du moment où ptr contient l'adresse de la variable i, *ptr et i correspondent au même emplacement mémoire, i et *ptr sont identiques, c'est le même objet. Donc afficher *ptr, c'est afficher i, la valeur 23.

(3) La valeur 55 est affectée à *ptr, c'est-à-dire à i et i vaut maintenant 55.

(4) La valeur 777 est affectée à la variable i et la valeur de l'expression *ptr est également 777.

Déclaration de pointeur

```
int *p;
```

signifie que *p est un int.

```
typedef struct{ ...}t_test *p;
```

signifie que *p est un structure t_test

```
double*p;
```

signifie que *p est un double, etc.

Ce point est très utile pour décrypter des écritures complexes avec des pointeurs à plusieurs étoiles.
Par exemple :

```
int **p;
```

signifie que *p est un int* et que **p est un int.

variable pouvant contenir l'adresse d'une variable qui contient elle même une adresse

Opérateur flèche : ->

Il concerne uniquement les structures

Soit la définition d'un type de structure nommé "ennemi" :

```
typedef struct{
    int vie;
    float x,y,dx,dy;
    int color;
}ennemi;
```

Une variable du type ennemi :

```
ennemi S1;
```

Une variable pointeur de type ennemi* :

```
ennemi*p;
```

avec

```
p=&S1;
```

Opérateur flèche : ->

Si on utilise l'opérateur étoile pour accéder aux champs de la structure S1 à partir du pointeur p, on a :

```
(*p).vie=1;
(*p).x=rand()%80;
(*p).y=rand()%25;
etc.
```

Afin d'éviter des erreurs et de faciliter l'utilisation des pointeurs de structure l'expression p-> est une notation abrégée de (*p). Et pour accéder aux champs de la structure ennemie, on écrit simplement :

```
p->vie=1;
p->x=rand()%80;
p->y=rand()%25;
etc.
```

Opérateur crochet : []

Un tableau est l'adresse mémoire du premier élément du tableau :

```
int tab[50];        // tab vaut &tab[0]
```

Puisqu'un tableau est une adresse mémoire il est possible de l'affecter à un pointeur :

```
int *p;
    p=tab;
```

ce qui signifie ensuite que tab et p sont équivalents et il est possible d'utiliser p à la place de tab :

```
int i;
    for (i=0; i<50; i++)
        p[i] = rand()%256;
```

et nous pouvons écrire également :

```
for (i=0; i<50; i++)
    *(p+i) = rand()%256;
```

Allocation dynamique de mémoire : fonction malloc()

Un pointeur permet aussi d'obtenir dynamiquement, pendant le fonctionnement du programme, une adresse mémoire allouée pour une variable de n'importe quel type

```
void* malloc(size_t taille);
```

Cette fonction de la bibliothèque stdlib.h alloue un bloc de mémoire de taille octets passée en argument et retourne l'adresse mémoire de la zone allouée (ou NULL si la mémoire est insuffisante).

void* est un pointeur générique, c'est-à-dire un pointeur qui peut fonctionner avec n'importe quel type d'objet.

La fonction malloc() qui est utilisée pour n'importe quel type d'objet, int*, char*, float*, double* ainsi que pour des adresses mémoire de structures créées par le programmeur. La fonction doit pouvoir allouer de la mémoire pour n'importe quel type d'objet.. C'est pourquoi la fonction malloc() renvoie un pointeur générique

Allocation dynamique de mémoire : fonction malloc()

La taille d'une zone mémoire pour un objet quelconque de type T s'obtient tout simplement avec l'opérateur sizeof (T).

Ainsi, soit ptr un pointeur de type T*, l'appel de la fonction est :.

```
ptr = malloc (sizeof( T ));
```

On a alors dans ptr l'adresse d'une zone mémoire de taille suffisante réservée pour un objet de type T

Libérer la mémoire allouée : la fonction free()

La mémoire allouée est réservée, c'est-à-dire inutilisable pour autre chose dans le programme, c'est pourquoi il est important de la libérer lorsque l'objet alloué n'est plus utilisé dans le programme afin de rendre cette mémoire à nouveau disponible. Cette opération inverse de l'allocation s'effectue avec un appel à la fonction :

```
void free(void*p);
```

Par exemple pour libérer la mémoire allouée au pointeur de l'exemple précédent l'appel ci-dessous est nécessaire :

```
free(ptr);
```

La valeur NULL

C’est une valeur spéciale de pointeur qui vaut en fait 0.

Le C garantit que 0 ne sera jamais une adresse utilisable par un objet quelconque. Et 0, qui n’est donc pas une adresse mémoire, est le seul cas où un entier peut être affecté à un pointeur.

Il est fréquent de rencontrer des tests de comparaison d’un pointeur avec la valeur NULL.

Par exemple, si la fonction malloc() ne peut pas allouer suffisamment de mémoire elle renvoie NULL et il est possible de garantir que le programme fonctionnera même si la mémoire est insuffisante avec un test sur la valeur de retour :

```
ptr=(int*)malloc(sizeof(int));
if (ptr!=NULL){
    printf("malloc réussi, le pointeur peut être utilisé\n");
    *ptr=rand()%300;
}
else
    printf("pas assez de mémoire,"
    "le pointeur ne peut pas être utilisé\n");
```